Unit -2 (Software Engineering)

Software Engineering

LECTURE NOTE – 9

SOFTWARE PROJECT MANAGEMENT

The job pattern of an IT company engaged in software development can be seen split in two parts:

- Software Creation
- Software Project Management

A project is well-defined task, which is a collection of several operations done in order to achieve a goal (for example, software development and delivery). A Project can be characterized as:

- Every project may have a unique and distinct goal.
- Project is not a routine activity or day-to-day operation.
- Project comes with a start and end time.
- Project ends when its goal is achieved. Hence, it is a temporary phase in the lifetime of an organization.
- Project needs adequate resources in terms of time, manpower, finance, material, and knowledge-bank.

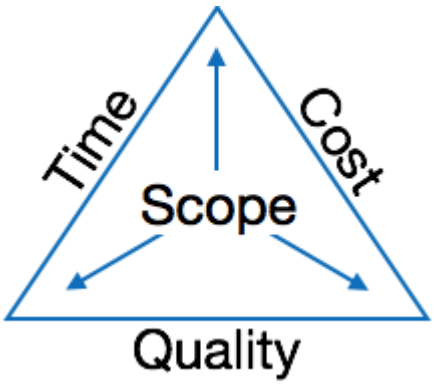
Software Project

A Software Project is the complete procedure of software development from requirement gathering to testing and maintenance, carried out according to the execution methodologies, in a specified period of time to achieve intended software product.

Need of software project management

Software is said to be an intangible product. Software development is a kind of all new stream in world business and there is very little experience in building

software products. Most software products are tailor made to fit client's requirements. The most important is that the underlying technology changes and advances so frequently and rapidly that the experience of one product may not be applied to the other one. All such business and environmental constraints bring risk in software development hence it is essential to manage software projects efficiently.



The image above shows triple constraints for software projects. It is an essential part of software organization to deliver quality product, keeping the cost within client's budget constrain and deliver the project as per scheduled. There are several factors, both internal and external, which may impact this triple constrain triangle. Any of the three factors can severely impact the other two.

Therefore, software project management is essential to incorporate user requirements along with budget and time constraints.

Software Management Activities

Software project management comprises of a number of activities, which contains planning of project, deciding scope of software product, estimation of cost in various terms, scheduling of tasks and events, and resource management. Project management activities may include:

- Project Planning
- Scope Management
- Project Estimation

Project Planning

Software project planning is task, which is performed before the production of software actually starts. It is there for the software production but involves no concrete activity that has any direct connection with the software production; rather it is a set of multiple processes, which facilitates software production. Project planning may include the following:

Scope Management

It defines scope of the project; this includes all the activities, process need to be done in order to make a deliverable software product. Scope management is essential because it creates boundaries of the project by clearly defining what would be done in the project and what would not be done. This makes project to contain limited and quantifiable tasks, which can easily be documented and in turn avoids cost and time overrun.

During Project Scope management, it is necessary to -

- Define the scope
- Decide its verification and control
- Divide the project into various smaller parts for ease of management.
- Verify the scope
- Control the scope by incorporating changes to the scope

Project Estimation

For an effective management, accurate estimation of various measures is a must. With the correct estimation, managers can manage and control the project more efficiently and effectively.

Project estimation may involve the following:

- **Software size estimation**

  Software size may be estimated either in terms of KLOC (Kilo Line of Code) or by calculating number of function points in the software. Lines of code depend upon coding practices. Function points vary according to the user or software requirement.

- **Effort estimation**

  The manager estimates efforts in terms of personnel requirement and man-hour required to produce the software. For effort estimation software size should be known. This can either be derived by manager's experience, historical data of organization, or software size can be converted into efforts by using some standard formulae.

- **Time estimation**

  Once size and efforts are estimated, the time required to produce the

software can be estimated. Efforts required is segregated into sub categories as per the requirement specifications and interdependency of various components of software. Software tasks are divided into smaller tasks, activities or events by Work Breakthrough Structure (WBS). The tasks are scheduled on day-to-day basis or in calendar months.

The sum of time required to complete all tasks in hours or days is the total time invested to complete the project.

- **Cost estimation**

This might be considered as the most difficult of all because it depends on more elements than any of the previous ones. For estimating project cost, it is required to consider -

- Size of the software
- Software quality
- Hardware
- Additional software or tools, licenses etc.
- Skilled personnel with task-specific skills
- Travel involved
- Communication
- Training and support

.

LECTURE NOTE – 10

PROJECT ESTIMATION TECHNIQUES

Lines of Code (LOC):

As the name suggests, LOC counts the total number of lines of source code in a project. The units of LOC are:

- KLOC- Thousand lines of code
- NLOC- Non-comment lines of code
- KDSI- Thousands of delivered source instruction

The size is estimated by comparing it with the existing systems of the same kind. The experts use it to predict the required size of various components of software and then add them to get the total size.

It's tough to estimate LOC by analyzing the problem definition. Only after the whole code has been developed can accurate LOC be estimated. This statistic is of little utility to project managers because project planning must be completed before development activity can begin.

Two separate source files having a similar number of lines may not require the same effort. A file with complicated logic would take longer to create than one with simple logic. Proper estimation may not be attainable based on LOC.

The length of time it takes to solve an issue is measured in LOC. This statistic will differ greatly from one programmer to the next. A seasoned programmer can write the same logic in fewer lines than a newbie coder.

Advantages:

- Universally accepted and is used in many models like COCOMO.
- Estimation is closer to the developer's perspective.
- Both people throughout the world utilize and accept it.
- At project completion, LOC is easily quantified.
- It has a specific connection to the result.
- Simple to use.

Disadvantages:

- Different programming languages contain a different number of lines.
- No proper industry standard exists for this technique.
- It is difficult to estimate the size using this technique in the early stages of the project.
- When platforms and languages are different, LOC cannot be used to normalize.

LECTURE NOTE – 11

Function Point Analysis:

In this method, the number and type of functions supported by the software are utilized to find FPC(function point count). The steps in function point analysis are:

- Count the number of functions of each proposed type.
- Compute the Unadjusted Function Points(UFP).
- Find the Total Degree of Influence(TDI).
- Compute Value Adjustment Factor(VAF).
- Find the Function Point Count(FPC).

The explanation of the above points is given below:

- **Count the number of functions of each proposed type:** Find the number of functions belonging to the following types:
- External Inputs: Functions related to data entering the system.
- External outputs: Functions related to data exiting the system.
- External Inquiries: They lead to data retrieval from the system but don't change the system.
- Internal Files: Logical files maintained within the system. Log files are not included here.
- External interface Files: These are logical files for other applications which are used by our system.
- **Compute the Unadjusted Function Points(UFP):** Categorise each of the five function types like simple, average, or complex based on their complexity. Multiply the count of each function type with its weighting factor and find the weighted sum. The weighting factors for each type based on their complexity are as follows:

Function type	Simple	Average	Complex
External Inputs	3	4	6

Function type	Simple	Average	Complex
External Output	4	5	7
External Inquiries	3	4	6
Internal Logical Files	7	10	15
External Interface Files	5	7	10

- **Find Total Degree of Influence:** Use the '14 general characteristics' of a system to find the degree of influence of each of them. The sum of all 14 degrees of influence will give the TDI. The range of TDI is 0 to 70. The 14 general characteristics are: Data Communications, Distributed Data Processing, Performance, Heavily Used Configuration, Transaction Rate, On-Line Data Entry, End-user Efficiency, Online Update, Complex Processing Reusability, Installation Ease, Operational Ease, Multiple Sites and Facilitate Change.
Each of the above characteristics is evaluated on a scale of 0-5.
- **Compute Value Adjustment Factor(VAF):** Use the following formula to calculate VAF

```
VAF = (TDI * 0.01) + 0.65
```

- **Find the Function Point Count:** Use the following formula to calculate FPC

```
FPC = UFP * VAF
```

Advantages:
- It can be easily used in the early stages of project planning.
- It is independent of the programming language.
- It can be used to compare different projects even if they use different technologies(database, language, etc).

Disadvantages:
- It is not good for real-time systems and embedded systems.
- Many cost estimation models like COCOMO use LOC and hence FPC must be converted to LOC.

LECTURE NOTE – 12

Effort Estimation in Project Management

Effort estimation is a crucial aspect of project management, playing a significant role in setting realistic timelines and allocating resources efficiently. It involves predicting the amount of time and effort required to complete a particular task or project.

What is effort estimation?

Effort estimation is the process of forecasting how much effort is required to develop or maintain a software application. This effort is traditionally measured in the hours worked by a person, or the money needed to pay for this work.

Effort estimation is used to help draft project plans and budgets in the early stages of the software development life cycle. This practice enables a project manager or product owner to accurately predict costs and allocate resources accordingly.

Effort estimation in Agile

Though effort estimation can be used in a traditional software development approach, it is more commonly associated with the Agile methodology. Here, the product owner must manage a list of project deliverables, known as a backlog. They will estimate the effort required to complete each item. Rather than using time or cost estimates, they will look at user stories and story points.

- A user story is a tool used to describe a software feature from the perspective of the consumer
- A story point is a unit that measures the amount of work in implementing a user story, taking into account the level of difficulty involved and the potential risk

  A product owner will compare the features of their new project with a previous one to determine the complexity of their user story and assign suitable story points.

Agile effort estimation techniques

There are many different Agile effort estimation techniques to choose from. Here are three of the most common ones:

1. **Planning Poker:** In this method, team members sit together in a circle to assign values to story points. Each individual will have a set of cards with the numerical values that can be assigned: 0, 1, 2, 3, 5, 8, 13, 20, 40, and 100. The product owner will read out a user story to the team members. They will have a discussion and then decide which value it should have. If everyone is in agreement, the final estimate is decided. If not, the team will discuss further until a consensus is reached.
2. **T-shirt Sizes:** Here, story points take the form of sizes: extra-small (XS), small (S), medium (M), large (L), and extra-large (XL). Estimators will determine the sizes to get a quick and rough estimate that can be converted to numbers later.
3. **Dot voting:** This approach enables team members to sort items in the product backlog from low to high priority. User stories are posted on a board and estimators get four or five dots to use as votes. The one with the most dots is deemed the highest-priority item, and so on.

Why is effort estimation important?

According to Forbes, estimation is "an investment that you should factor into software development efforts of any substance."

Effort estimation is vital to the software development process as it helps teams to ensure a product is developed and delivered on time. For product owners specifically, effort estimation enables them to manage resources.

LECTURE NOTE – 13

COCOMO Model

Cocomo (Constructive Cost Model) is a regression model based on LOC, i.e **number of Lines of Code**. It is a procedural cost estimate model for software projects and is often used as a process of reliably predicting the various parameters associated with making a project such as size, effort, cost, time, and quality. It was proposed by Barry Boehm in 1981 and is based on the study of 63 projects, which makes it one of the best-documented models.
The key parameters which define the quality of any software products, which are also an outcome of the Cocomo are primarily Effort & Schedule:

- **Effort:** Amount of labor that will be required to complete a task. It is measured in person-months units.
- **Schedule:** Simply means the amount of time required for the completion of the job, which is, of course, proportional to the effort put in. It is measured in the units of time such as weeks, and months.
Different models of Cocomo have been proposed to predict the cost estimation at different levels, based on the amount of accuracy and correctness required. All of these models can be applied to a variety of projects, whose characteristics determine the value of the constant to be used in subsequent calculations.

These characteristics pertaining to different system types are mentioned below. Boehm's definition of organic, semidetached, and embedded systems:

1. Organic – A software project is said to be an organic type if the team size required is adequately small, the problem is well understood and has been solved in the past and also the team members have a nominal experience regarding the problem.
2. Semi-detached – A software project is said to be a Semi-detached type if the vital characteristics such as team size, experience, and knowledge of the various programming environment lie in between that of organic and Embedded. The projects classified as Semi-Detached are comparatively less familiar and difficult to develop compared to the organic ones and require more experience and better guidance and creativity. Eg: Compilers or different Embedded Systems can be considered Semi-Detached types.
3. Embedded – A software project requiring the highest level of complexity, creativity, and experience requirement fall under this category. Such software requires a larger team size than the other two models and also the developers need to be sufficiently experienced and creative to develop such complex models.

1. Basic COCOMO Model
2. Intermediate COCOMO Model
3. Detailed COCOMO Model

1. **Basic Model –.**
2. The effort and the time taken to complete both are calculated through predefined equations.
3. The formula used is-

4. ***Effort(E)= a*(KLOC)^b PM(Person month)***
   Time = c(E)d
   Person Required = effort/time
5. Where,
   KLOC = size of the product i.e. no. of lines of code.
   a, b, c, d are constants and have different values for different models.
   Time is the time required to develop the product and the unit is months.
   And the effort is the total effort calculated that will be required to complete the work calculated in PM (Person Months).
6. If the constants are replaced by actual values, then the equations will be-
7. Organic-
   E = 2.4(KLOC) 1.05 PM
   T = 2.5(E) 0.38 Months
8. Semi-detached-
   E= 3.0(KLOC) 1.12 PM
   T = 2.5(E) 0.35 Months
9. Embedded-
   E= 3.6(KLOC) 1.20 PM
   T= 2.5(E) 0.32 Months
10. This is how the effort and time taken to complete the project are calculated.

The above formula is used for the cost estimation of for the basic COCOMO model, and also is used in the subsequent models. The constant values a,b,c, and d for the Basic Model for the different categories of the system:

Software Projects	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

The effort is measured in Person-Months and as evident from the formula is dependent on Kilo-Lines of code. The development time is measured in months. These formulas are used as such in the Basic Model calculations, as not much consideration of different factors such as reliability, and expertise is taken into account, henceforth the estimate is rough.

Below is the C++ program for Basic COCOMO.

```
// C++ program to implement basic COCOMO


#include <bits/stdc++.h>
```

```
using namespace std;



// Function

// For rounding off float to int

int fround(float x)

{

    int a;

    x = x + 0.5;

    a = x;

    return (a);

}



// Function to calculate parameters of Basic
COCOMO

void calculate(float table[][4], int n, char
mode[][15],

               int size)

{

    float effort, time, staff;



    int model;
```

```
    // Check the mode according to size


    if (size >= 2 && size <= 50)

        model = 0; // organic


    else if (size > 50 && size <= 300)

        model = 1; // semi-detached


    else if (size > 300)

        model = 2; // embedded


    cout << "The mode is " << mode[model];


    // Calculate Effort

    effort = table[model][0] * pow(size,
table[model][1]);


    // Calculate Time

    time = table[model][2] * pow(effort,
table[model][3]);


    // Calculate Persons Required

    staff = effort / time;
```

```
    // Output the values calculated

    cout << "\nEffort = " << effort << " Person-
Month";



    cout << "\nDevelopment Time = " << time << "
Months";



    cout << "\nAverage Staff Required = " <<
fround(staff)

         << " Persons";

}



int main()

{

    float table[3][4] = { 2.4, 1.05, 2.5, 0.38,
3.0, 1.12,

                          2.5, 0.35, 3.6, 1.20,
2.5, 0.32 };



    char mode[][15]

        = { "Organic", "Semi-Detached",
"Embedded" };



    int size = 4;
```

```
    calculate(table, 3, mode, size);



    return 0;

}
```

Output:

```
The mode is Organic

Effort = 10.289 Person-Month

Development Time = 6.06237 Months

Average Staff Required = 2 Persons
```

2. Intermediate Model – The basic Cocomo model assumes that the effort is only a function of the number of lines of code and some constants evaluated according to the different software systems. However, in reality, no system's effort and schedule can be solely calculated on the basis of Lines of Code. For that, various other factors such as reliability, experience, and Capability. These factors are known as Cost Drivers and the Intermediate Model utilizes 15 such drivers for cost estimation. Classification of Cost Drivers and their Attributes:

(i) Product attributes –

- Required software reliability extent
- Size of the application database
- The complexity of the product
- Run-time performance constraints
- Memory constraints
- The volatility of the virtual machine environment
- Required turnabout time
- Analyst capability
- Software engineering capability
- Applications experience
- Virtual machine experience
- Programming language experience
- Use of software tools
- Application of software engineering methods
- Required development schedule

3. Detailed Model – Detailed COCOMO incorporates all characteristics of the intermediate version with an assessment of the cost driver's impact on each step of the software engineering process. The detailed model uses different effort multipliers for each cost driver attribute. In detailed cocomo, the whole software is divided into different modules and then we apply COCOMO in different modules to estimate effort and then sum the effort. The Six phases of detailed COCOMO are:

1. Planning and requirements
2. System design
3. Detailed design
4. Module code and test
5. Integration and test
6. Cost Constructive model

Advantages of the COCOMO model:

1. Provides a systematic way to estimate the cost and effort of a software project.
2. Can be used to estimate the cost and effort of a software project at different stages of the development process.
3. Helps in identifying the factors that have the greatest impact on the cost and effort of a software project.
4. Can be used to evaluate the feasibility of a software project by estimating the cost and effort required to complete it.

Disadvantages of the COCOMO model:

1. Assumes that the size of the software is the main factor that determines the cost and effort of a software project, which may not always be the case.
2. Does not take into account the specific characteristics of the development team, which can have a significant impact on the cost and effort of a software project.
3. Does not provide a precise estimate of the cost and effort of a software project, as it is based on assumptions and averages.

LECTURE NOTE – 14

RISK ANALYSIS

Risk management involves all activities pertaining to identification, analyzing and making provision for predictable and non-predictable risks in the project. Risk may include the following:

- Experienced staff leaving the project and new staff coming in.
- Change in organizational management.
- Requirement change or misinterpreting requirement.
- Under-estimation of required time and resources.
- Technological changes, environmental changes, business competition.

Risk Management Process

There are following activities involved in risk management process:

- **Identification -** Make note of all possible risks, which may occur in the project.
- **Categorize -** Categorize known risks into high, medium and low risk intensity as per their possible impact on the project.
- **Manage -** Analyze the probability of occurrence of risks at various phases. Make plan to avoid or face risks. Attempt to minimize their side-effects.
- **Monitor -** Closely monitor the potential risks and their early symptoms. Also monitor the effective steps taken to mitigate or avoid them.

LECTURE NOTE – 15

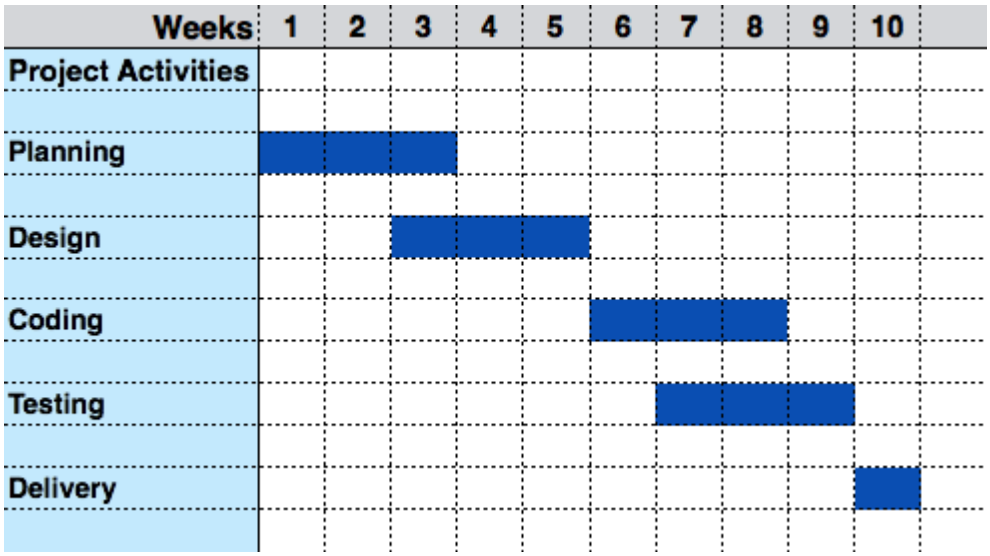
Software Project Scheduling

A schedule in your project's time table actually consists of sequenced activities and milestones that are needed to be delivered under a given period of time.

Project schedule simply means a mechanism that is used to communicate and know about that tasks are needed and has to be done or performed and which organizational resources will be given or allocated to these tasks and in what time duration or time frame work is needed to be performed. Effective project scheduling leads to success of project, reduced cost, and increased customer satisfaction. Scheduling in project management means to list out activities, deliverables, and milestones within a project that are delivered. It contains more notes than your average weekly planner notes.

Gantt Chart

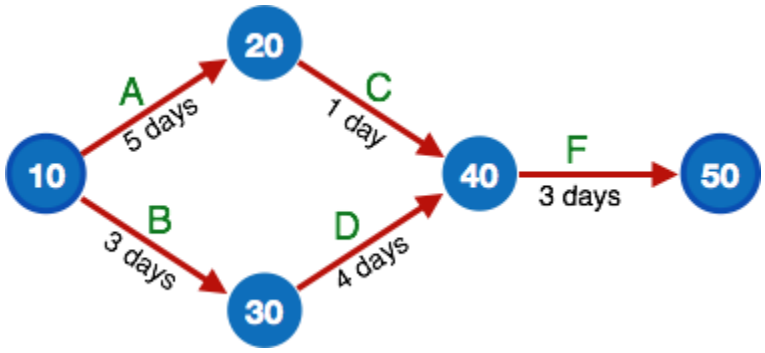
Gantt chart was devised by Henry Gantt (1917). It represents project schedule with respect to time periods. It is a horizontal bar chart with bars representing activities and time scheduled for the project activities.



LECTURE NOTE – 16

PERT Chart

Program Evaluation & Review Technique) (PERT) chart is a tool that depicts project as network diagram. It is capable of graphically representing main events of project in both parallel and consecutive ways. Events, which occur one after another, show dependency of the later event over the previous one.



Events are shown as numbered nodes. They are connected by labeled arrows depicting the sequence of tasks in the project.